

Traffic Light AI Usage Framework

This coursework uses a Traffic Light AI Usage Framework to guide how you may or may not use generative AI tools (e.g., ChatGPT, Copilot, etc.). The colour assigned to this task indicates the level of permitted AI assistance. You are expected to follow these guidelines carefully and ensure all submitted work reflects your own understanding and academic integrity.



Red - no generative AI is permissible

Yellow - some generative AI is permitted

Green - generative AI is expected

Please check the assigned colour for this coursework and adhere to the corresponding requirements outlined below.

● Red - Prohibit to use gen AI

You must not use any generative AI tools at any stage of this assignment. All work must be entirely your own. Any use of AI assistance will be considered a breach of academic integrity.

● Yellow - Restricted usage of gen AI

Limited use of generative AI is allowed for specific purposes only (e.g., brainstorming ideas, clarifying concepts, or language refinement). AI must not be used to generate substantive content for submission. You must clearly acknowledge any AI use according to the course guidelines.

● Green - Permitted to use gen AI

You may use generative AI tools to support your work, including idea generation, drafting, and editing. However, you remain responsible for the accuracy, originality, and integrity of your submission. Proper disclosure of how AI was used is required.

AI Usage Level for This Coursework: ● Yellow (Restricted Use)

Guidelines for the Responsible Use of Generative AI (GenAI)

You may use generative AI (GenAI) tools to support learning and development in this assignment in the following ways:

- Concept Clarification: Explaining standard data structures array, linked-list, sorting and searching algorithm.
- Syntax & Toy Examples: Generating isolated, small code snippets to learn C++ syntax or basic library usage. It is **must not** related to the assignment's specific problem logic.
- Debugging Assistance: Identifying syntax errors, exceptions, or understanding generic runtime error messages in code you wrote yourself.
- Brainstorming: Discussing high-level approaches or exploring edge cases to consider when designing your data structures.

However, the following are not permitted:

- AI-generated content must not be submitted *as-is* as your own work. You are expected to understand, adapt, and significantly modify any AI-generated output.
- GenAI must not be used to complete the assessment in full, bypass problem-solving processes, or replace your own reasoning and design decisions.
- Core Logic & Algorithm Generation: You must not use GenAI to design, implement, or write the core algorithms required by the assignment
- Automated Data Structure Tracing: GenAI must not be used to generate step-by-step trace tables, state diagrams, or dry-run outputs required in your submission.
- Complexity & Performance Analysis: You must not use GenAI to analyse or write Big-O asymptotic time or space complexity justifications for your custom implementations.

COMPARATIVE ANALYSIS OF ARRAYS AND SINGLY LINKED LISTS FOR SORTING AND SEARCHING IN SMART HEALTHCARE & HOSPITAL PATIENT MANAGEMENT SYSTEM

(Working Duration: Friday of Week 4 to Friday of Week 9 – 30 Marks)

Background

MetroHealth System is a major healthcare provider experiencing a surge in daily patient visits across its network. Each day, thousands of patients enter different facility types to seek consultation, treatment, emergency care, or routine health screenings. Concerned about waiting times, bed allocation, and operational bottlenecks, the hospital board wishes to optimize healthcare resource management. To better understand patient flow and resource usage, detailed registration and clinical data were collected across three distinct facilities:

1. Facility A (General Hospital - Metropolitan Hub):
 - Adult and elderly patients admitted or visited for outpatient consultations, emergency care, surgical procedures, or routine check-ups.
2. Facility B (University Medical Center):
 - Young adults, students, and pediatric patients undergoing preventive health screenings, minor treatments, vaccination drives, or sports injury rehab.
3. Facility C (Community Health Clinic - Rural/Suburban):
 - Residents of all ages seeking general practitioner consultations, chronic disease monitoring, and maternal care.

The dataset collected for each patient includes:

1. Patient ID - A unique identifier for each patient (example: PT10042).
2. Age - The age of the patient.
3. Care Type - Type of healthcare service received (Emergency, Outpatient, Inpatient, Vaccination, Rehabilitation, Routine Checkup).
4. Length of Stay / Visit Duration (Hours) - The total hours spent in the facility.
5. Base Cost Per Hour (MYR/hr) - The standard billing rate associated with the specific care type.
6. Days Visits Per Year - The average number of visits or days per year the patient utilizes hospital services.

Hospital administrators aim to analyze this data to identify high-cost care departments, compare medical service utilization across facilities, and design policies that reduce patient wait times while optimizing medical resource distribution.

To perform this analysis efficiently, the hospital's IT research team decided to store and process the dataset using arrays and singly linked lists, applying various sorting and searching algorithms. By comparing the performance of these fundamental data structures and algorithms, the administration can determine the most effective methods to manage large medical records while gaining actionable clinical insights.

Assignment Task

1. Group Formation

- a. This assignment must be completed in teams, with a minimum of **FOUR (4)** members and a maximum of **FIVE (5)** members per team.

2. Datasets Provided

- a. Three datasets are provided for this assignment:
 - Dataset 1 - Facility A (General Hospital): Adult & elderly patients using Emergency, Inpatient and Outpatient care.
 - Dataset 2 - Facility B (University Medical Center): Students & young adults using Rehabilitation, and Vaccination.
 - Dataset 3 - Facility C (Community Health Clinic): Residents of all ages using Routine Checkup.

3. Program Implementation

- a. Implement Dataset 1, Dataset 2, and Dataset 3 using arrays.
- b. Implement the same datasets using singly linked lists.

- c. Ensure the data structures allow easy access for sorting, searching, and analysis operations.
- d. Develop two (2) separate programs:
 - o One focusing strictly on array implementation.
 - o The other focuses strictly on singly linked list implementation.

4. Patient Demographic & Billing Categorization

- a. Recategorize patients into the following age groups:
 - o 0–17: Pediatrics & Adolescents
 - o 18–25: Young Adults / University Students
 - o 26–45: Working Adults (Early Career)
 - o 46–60: Working Adults (Late Career)
 - o 61–100: Senior Citizens / Geriatric Care
- b. For each age group, determine:
 - o Most preferred / requested Care Type.
 - o Total medical cost produced (Cost = Length of Stay x Base Cost Per Hour x Days Visits Per Year).
- c. Average medical cost per patient:

$$\text{Average Cost} = \frac{\text{Total Medical Cost in Age Group}}{\text{Number of Patients in Age Group}}$$

5. Healthcare Expenditure & Service Analysis.

- a. Calculate total medical billing costs per dataset.
- b. Determine total medical costs grouped by Care Type.
- c. Compare expenditure and visit durations across datasets and age groups.
- d. Display results as text-based tables in the console.

Sample:

Age Group: 18–25 (Young Adults / University Students)			
Care Type	Patient Count	Total Cost (\$)	Average Cost per Patient (\$)
Vaccination	50	2,500.00	50.00
Rehabilitation	30	4,500.00	150.00
Emergency	25	12,000.00	480.00
Outpatient	20	6,000.00	300.00
Total Billing for Age Group: \$25,000.00			

6. Sorting Experiments

- a. Sort datasets by Age, Visit Duration (Length of Stay), or Total Medical Cost.
- b. Implement at least one (1) sorting algorithm.
- c. Compare time complexity and memory usage between arrays and singly linked lists.
- d. Display sorting performance results as text-based tables in the console.

7. Searching Experiments

- a. Search for patient records matching specific criteria, such as:
 - Age group (example: 61–100)
 - Specific Care Type (example: Emergency or Rehabilitation)
 - Visit duration threshold (example: > 24 hours)

- b. Implement search algorithms on unsorted vs. sorted data.
- c. Compare execution time, steps/comparisons, and memory overhead between arrays and singly linked lists.
- d. Display search query results in formatted console tables.

8. Performance Analysis

- a. Record execution time and memory footprint for all sorting and searching operations.
- b. Analyze under which conditions arrays or singly linked lists perform better (example: contiguous memory access vs. node pointer overhead, dynamic insertion vs. fixed indexing).
- c. Discuss real-world advantages and trade-offs of each data structure in medical record management.

9. Clinical Insights and Recommendations

- a. Compare treatment costs and service preferences across datasets and age groups using text table.
- b. Identify which demographic groups generate the highest healthcare billing and which care types experience the highest patient traffic.
- c. Provide practical recommendations for hospital administrators to optimize patient admission pipelines and reduce healthcare bottlenecks based on demographic trends.

AI USAGE POLICY FOR CODING ASSIGNMENTS

1. GenAI Declaration requirement

All use of GenAI tools must be clearly declared and properly acknowledged in the submission, and students are responsible for verifying the correctness and appropriateness of any AI assisted output.

2. Students must be able to *explain and justify every part of their code* during evaluation or questioning.

3. If AI tools are **used for assistance**, students *must clearly declare the purpose of AI usage*

Example, "AI was used to identify appropriate keywords related to file handling in C++"

4. **Copying, paraphrasing, or adapting AI-generated code or solutions**, whether modified or unmodified, is **considered a violation of academic integrity**.

5. **Any misuse of AI tools**, including generating solutions beyond permitted assistance, **may result in penalties** including:

- **Zero (0) marks** for the assignment **OR**
- **Disciplinary action** in accordance with institutional policies.

Submission Guidelines #1: Program and Video Submission – Lab Work #1 (15 Marks)

1. Your team is required to utilize C++ programming to develop this prototype, comprising two programs in this section.
 - Do not use GUI, charts, or external visualization libraries.
 - Organize data neatly in text tables using formatting (e.g., fixed-width columns, separators).

2. **Built-in containers such as <list>, <vector>, etc. are not allowed** in this assignment. All containers need to be self-created.

Refer to the link: <https://www.geeksforgeeks.org/containers-cpp-stl/> for further information on built-in containers in STL C++.

3. The program will be enhanced if your team effectively implements and compares various useful searching or sorting algorithms in your developed prototype.
4. The evaluation criteria for this lab work #1 include assessing the clarity and structural design of the code, as well as the quality of comments and adherence to good programming practices. (e.g., indentation, meaningful identifier names, comments, etc.).
5. **Creativity is encouraged.**
The best solution is not limited to the core features specified in this assignment. Teams are encouraged to extend the system with additional logical functionalities, provided that all enhancements are clearly related to and supported by data structure concepts.
6. **Plagiarism Policy and Penalty.**
Plagiarism in any form, including copying code, algorithms, documentation, or analysis

from other groups, seniors, online sources, or AI-generated content without proper acknowledgment, is strictly prohibited.

*Any group found to have committed plagiarism will receive a **FAIL (0 marks) for the assignment**. Further disciplinary action may be taken in accordance with the institution's academic integrity policy.*

All submitted work must be original and developed solely by the assigned group members.

7. The team leader must submit the system solution in a ZIP file format to Moodle by Friday of Week 9, no later than 5:00 pm.
 - a) Include only the .cpp, .hpp, and .csv/text files in the ZIP file.
 - b) The ZIP file must follow this naming format:

`"<GroupNo>_<Team Leader ID>_<Member 1 ID>_<Member 2 ID>_<Member 3 ID>.zip"`

For example, **"G1_TP012345_TP014556_TP067554_TP034325.zip"**

8. Each team leader must also upload **ONE (1)** video recording to the Moodle system by Friday of Week 9, no later than 5:00 pm.
 - a) The maximum duration for the overall video recording is 30 minutes.
 - b) All members must participate in the video recording, presenting and explaining the development process in detail, with each member speaking for no more than 5 minutes.
 - c) Each member must relate their explanation to the workload matrix distribution table provided in the Word document.
 - d) Compress the final video recording to under 200MB before submitting.
 - e) If the video exceeds the specified time limit (30 minutes), it will only be assessed up to the specified duration.
 - f) Videos must be recorded at normal speed (1x) and cannot be sped up or adjusted to meet the demo video duration requirements.
 - g) The video recording file must adhere to the following name format:

`"<GroupNo>_<Team Leader ID>_<Member 1 ID>_<Member 2 ID>_<Member 3 ID>.mp4"`

For example, **"G1_TP012345_TP014556_TP067554_TP034325.mp4"**

9. Refer to **Page 8** for marking criteria of this Lab Evaluation Work #1 submission.

Submission Guidelines #2: Documentation Submission – Solution Work (15 Marks)

1. Create a **Word document** (refer Appendix 1 in page 9) that explains the overall efficiency and limitations of your developed system
2. Clearly demonstrate the differences in implementation and performance between arrays and linked lists based on the ***operations performed*** in this specific analysis scenario.
3. Present results that clearly show the advantages and disadvantages of using arrays and linked lists in this specific sentiment analysis scenario.
4. Include a summary of all your hypotheses, observations, experimental results, justifications, and reflections, etc. in your word documentation.
5. Your team must include a workload distribution matrix table after the cover page, showing each member's contributions to the project.

Note that this table will impact each member's personal final mark in Lab Evaluation Work #1 based on their stated contribution percentage.

6. If you use some code which has been taken or adapted from another source (book, magazine, internet, forum, etc.) then this must be **cited and referenced** using **APA Referencing Style within your source code**, and this must be mentioned explicitly in the **report**. Failure to reference code properly will be treated as plagiarism.
7. The team leader is required to upload a word document before / on Friday of Week 9,

no later than 5:00 pm.

The documentation should be named using format:

“<GroupNo>_<Team Leader ID>_<Member 1 ID>_<Member 2 ID>_<Member 3 ID>.docx”

For example, “G1_TP012345_TP014556_TP067554_TP034325.docx”



8. Refer to the **Page 8** for marking criteria of this Solution Work submission.

MARKING CRITERIA (Lab Evaluation Work #1 - 15 MARKS)

This Lab Evaluation Work #1 will be evaluated according to the following performance criteria:

Assessment Components	Inclusive	15 Marks
CLO3: Lab Evaluation Task #1 – 30-Minute Video Recording (Assessment will be based on individual performance)		
Practical Skills: Use of Data Structures & Algorithms + Personal Understanding		
Utilization of data structures	Technical Proficiency	
Implementation of relevant algorithms	Technical Proficiency	
Demonstrates understanding of data structures/algorithms used	Comprehensive Understanding	
Justifies choices of structures/algorithms	Insightful Justification	

MARKING CRITERIA (Solution Work - 15 MARKS)

This solution work will be evaluated according to the following performance criteria:

Assessment Components	Inclusive	15 Marks
-----------------------	-----------	----------

CLO2: Solution Work - Documentation (Assessment will be based on group component)		
Theoretical Explanation (e.g., Data Structures, Algorithms)	Clear explanation	
Input Output Screenshots	Adequate Screenshots	
Summary Discussions (Inclusive Time and Space Complexity)	Clear and insightful analysis	
Conclusion & Reflection (Other Relevant / Importance Discussions)	Clearly highlighted and insightful	
Content Organization	Well-structured and logical flow	

Approximation of Total Pages for the documentation: **40 (max)**.

Approximation of Words for the documentation: **6000 words (max)**

APPENDIX 1: SOLUTION WORK - DOCUMENTATION (15 MARKS)

The report outline as below:

- **Cover Page** (1 page)
 - Module Code and Name
 - Intake Code
 - Group Number
 - Member List.
- **Workload Matrix Table with signature** (1 page)
- **Theoretical Explanation** (2 - 4 pages)
 - Elaborate on the data manipulations in chosen data structures.
- **Input Output Screenshots** (5 - 15 pages)
 - System input output screenshots
- **Summary Discussions** (5 - 15 pages)
 - System efficiency (*including actual execution time, time efficiency, space efficiency etc.*).
 - Summarize and discuss all the observations / thoughts made during the development.
 - Critically summarize the strengths and weaknesses of the code you developed.
- **Conclusion and Reflection** (1 - 2 pages)
 - Summary of your works
 - Describe your future work in terms of your system weaknesses, if any.

- Additional personal thoughts on this assignment, if any.
- **References (if applicable)** (1 page)
- **Appendix (if applicable)** (1 page)

Approximation of Total Pages : **40 (max).**
Approximation of Words : **6000 words (max)**